

Cuestionario: Docker y Kubernetes

1. ¿Qué problema busca resolver la contenedorización?

Respuesta: Busca evitar diferencias entre entornos, empaquetando la aplicación con sus dependencias y configuraciones para que funcione de manera consistente en desarrollo, pruebas y producción.

2. ¿En qué se diferencia una imagen de un contenedor?

Respuesta: Una imagen es una plantilla inmutable que contiene la aplicación y sus dependencias; un contenedor es una instancia en ejecución creada a partir de esa imagen.

3. ¿Cuál es la secuencia básica para crear y ejecutar un contenedor con Docker?

Respuesta: Primero se escribe un Dockerfile, luego se construye una imagen y finalmente se ejecuta un contenedor a partir de esa imagen.

4. ¿Por qué los contenedores generalmente consumen menos recursos que las máquinas virtuales?

Respuesta: Porque comparten el kernel del sistema operativo anfitrión y no necesitan incluir un sistema operativo completo en cada instancia.

5. ¿Para qué se utiliza un Dockerfile?

Respuesta: Se utiliza para definir de manera reproducible las instrucciones necesarias para construir una imagen, como copiar archivos, instalar dependencias y establecer el comando de inicio.

6. ¿Se pueden crear varios contenedores utilizando una misma imagen?

Respuesta: Sí. Una misma imagen puede utilizarse para crear múltiples contenedores independientes.

7. ¿Qué elementos básicos necesita una imagen para convertirse en un contenedor?

Respuesta: La imagen debe ejecutarse mediante una plataforma o runtime de contenedores, como Docker Engine.

8. ¿Por qué administrar manualmente muchos contenedores puede resultar complicado?

Respuesta: Porque se debe controlar dónde se ejecutan, qué hacer cuando fallan, cómo distribuir el tráfico, cómo escalarlos y cómo actualizar sus versiones.

9. ¿Qué función principal cumple Kubernetes?

Respuesta: Automatiza el despliegue, escalado, actualización y administración de aplicaciones contenerizadas dentro de un clúster.

10. ¿Qué significa que Kubernetes trabaje con un “estado deseado”?

Respuesta: Significa que el usuario declara cómo debería funcionar la aplicación, por ejemplo, con tres réplicas, y Kubernetes intenta mantener continuamente esa condición.

11. Si un Deployment solicita tres réplicas y uno de sus Pods falla, ¿qué hará Kubernetes?

Respuesta: Detectará que solo quedan dos réplicas disponibles y creará un nuevo Pod para recuperar las tres solicitadas.

12. ¿Cómo se relacionan un clúster, un nodo, un Pod y un contenedor?

Respuesta: El clúster está formado por nodos; los nodos ejecutan Pods y dentro de los Pods funcionan uno o más contenedores relacionados.

13. ¿Cuál es la unidad mínima que Kubernetes despliega y administra?

Respuesta: El Pod

14. ¿Cuál es la función de un Deployment?

Respuesta: Define cómo debe ejecutarse una aplicación, cuántas réplicas debe mantener y cómo deben realizarse sus actualizaciones.

15. ¿Por qué se necesita un Service si ya existen Pods?

Respuesta: Porque los Pods pueden desaparecer, ser reemplazados o cambiar de dirección. El Service proporciona un punto de acceso lógico y estable para comunicarse con ellos.

16. ¿Qué significa que Kubernetes tenga capacidad de autorrecuperación?

Respuesta: Significa que puede reiniciar contenedores, reemplazar Pods fallidos o reprogramar cargas cuando detecta problemas.

17. ¿Qué sucede cuando aumenta considerablemente la cantidad de usuarios de una aplicación?

Respuesta: Kubernetes puede aumentar la cantidad de réplicas para distribuir la carga y, cuando la demanda disminuya, puede reducirlas.

18. ¿Docker y Kubernetes son tecnologías competidoras?

Respuesta: No necesariamente. Docker se utiliza principalmente para construir imágenes y ejecutar contenedores, mientras Kubernetes administra aplicaciones contenerizadas a escala.

19. ¿Cuál es la principal diferencia entre la función de Docker y la de Kubernetes?

Respuesta: Docker empaqueta y ejecuta aplicaciones en contenedores; Kubernetes despliega, escala y recupera aplicaciones contenerizadas.

21. ¿Qué diferencia existe entre un Deployment y un Service en Kubernetes?

Respuesta: El Deployment administra la ejecución, cantidad de réplicas y actualización de los Pods; el Service proporciona un punto de acceso estable para comunicarse con esos Pods.